

ClassSight: Smart Classroom Attendance & Verification System

Project Proposal for UCS503P

Submitted to: Jhonsy Bansal

Thapar Institute of Engineering and Technology

Manit Sharma, CSED*

Parv Rawat, CSED[†]

Guransh Rally, CSED[‡]

August 24, 2026

1 Higher-order goal

... secondary framing

This project aims to improve classroom efficiency and academic accountability by reducing the time and effort required to record and verify student attendance. While the underlying technique (face recognition) is broadly applicable, the immediate engineering objective is to translate *reliable, low-friction attendance capture* into a measurable, deployable software solution.

2 Time-to-value

... primary heuristic

- We will deliver meaningful increments quickly by prototyping the core teacher workflow (upload photo → verify → save) and validating it within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations to reduce release friction and surface integration issues early.
- Success is defined by what can be delivered and measured in the first iteration, then improved based on teacher feedback.

3 Problem Statement

Teachers in a typical classroom setting lose meaningful instructional time to manual roll-call, and paper- or spreadsheet-based attendance is prone to errors, proxy marking, and inconsistent record-keeping. Common pain points include:

- **Time loss:** manual roll-call consumes several minutes per class, compounding across sections and semesters.
- **Proxy attendance:** verbal or paper-based roll-call is easy to circumvent.
- **Fragmented records:** attendance data is scattered across registers/spreadsheets, making consolidated reporting difficult.

*Roll No: 1024030331, Email: msharma2_be24thapar.edu

[†]Roll No: 1024030265, Email: prawat_be24thapar.edu

[‡]Roll No: 1024030319, Email: grally_be24thapar.edu

- **No self-service visibility:** students cannot easily check their own attendance percentage or history.

Why this matters now (contextual relevance): With classroom sizes and course loads increasing, even a modest reduction in per-class administrative overhead compounds into significant recovered instructional time over a semester.

4 Proposed Solution

...Software Proposition

4.1 Overview

Build a **web-based** “Photo-to-Attendance” system with the following components:

- **Teacher capture workflow:** teacher starts a session and uploads a single classroom photograph.
- **Automated recognition:** the system matches detected faces against pre-registered student embeddings and returns confidence-scored candidates.
- **Manual verification:** teacher confirms, corrects, or manually adds students before the record is finalized.
- **Fallback correction via NFC tap:** any student missed by face recognition (poor angle, occlusion, low confidence) can self-mark by tapping their library card on a phone-based NFC panel during the same verification step, instead of the teacher manually searching for and adding them.
- **Status transparency:** students view their own attendance percentage and session-wise history.
- **Admin/analytics dashboard:** classroom- and course-level attendance summaries and exportable reports.

Face-recognition remains the primary attendance mechanism; the NFC tap is a lightweight, secondary path for closing gaps in the same session rather than a separate attendance system.

4.2 Core workflow

1. Teacher opens a classroom and starts an attendance session.
2. Teacher uploads a classroom photograph (JPG/PNG, up to 10 MB).
3. System runs face recognition and returns identified students with confidence scores.
- 3a. For any student not recognized, the student (or teacher) taps the library card on the NFC scan panel; the system looks up `UID` $\rightarrow$ `roll_number` and adds them to the current session’s attendance list alongside the recognized students.
4. Teacher reviews the combined results (recognized + NFC-added), marks remaining absentees, adds any student manually if needed, and saves the final record.

One-time enrollment: each student taps their card once during enrollment; the captured UID is mapped to their roll number by an admin/teacher and stored in the UID-mapping table.

4.3 Operational constraints for an educational setting

- Keep the solution usable on standard classroom devices (laptop/tablet/phone) via responsive web design.
- Avoid dependence on novel recognition research; use an established face-embedding approach and the browser-native Web NFC API, focusing on workflow, automation, and system correctness.
- Design for deployability using a single low-cost cloud instance suitable for a student project (AWS free-tier friendly).
- Web NFC scanning is currently supported only on Chrome for Android; since it is only used briefly for the occasional missed student during verification, a single designated Android phone is sufficient.

5 Solution Approach

... Engineering focus

This is an engineering course project, so we focus on the workflow, integration, and deployment aspects rather than novel algorithm research.

5.1 Recognition assistance

... non-research

Face matching will use an established, off-the-shelf approach:

- Pre-register each student's face embedding at enrollment time.
- Use an existing face-embedding library to detect and match faces in the uploaded photo against registered embeddings, with a confidence threshold.
- Present all matches (including low-confidence ones) to the teacher for decision; the system never finalizes attendance without teacher confirmation.

5.2 Fallback correction via NFC

... non-research

The NFC tap is implemented as a correction step inside the existing verification screen, using browser-native and standard backend primitives, with no dedicated NFC hardware beyond the students' existing library cards and a single Android phone:

- **Scanning:** the `NDEFReader` interface of the Web NFC API reads the card UID on tap; no native app is required.
- **Matching:** the backend performs a simple `UID` $\rightarrow$ `roll_number` lookup against a pre-populated enrollment table and adds the student to the *current session's* attendance list – no fuzzy matching or inference is involved.
- **Reflecting the tap:** the verification screen the teacher is already viewing re-fetches (or lightly polls) the session's attendance list so the newly tapped student appears alongside the recognized ones, without needing a separate live-dashboard view.
- **Feedback:** client-side detection of a successful tap triggers a browser `Notification` confirming the student was added, giving immediate feedback without disrupting the verification flow.

5.3 Web architecture

... web-based solution

A typical 3-tier web architecture:

- **Frontend:** responsive React UI for login, classroom/session management, photo upload, the verification screen (including the NFC fallback panel), and analytics.
- **Backend API:** FastAPI service handling authentication, session management, image upload, face recognition, NFC UID scan ingestion for fallback correction (POST `/api/session/{id}/scan`), and attendance confirmation.
- **Database:** PostgreSQL (or SQLite for early prototyping) storing users, classrooms, students, sessions, attendance records, and the UID-to-roll-number enrollment mapping; Amazon S3 for classroom photograph storage.

5.4 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run tests (authentication, attendance APIs, recognition functions) on every change.
- Produce repeatable deployment artifacts to a staging environment on EC2.
- Enable safe and frequent releases of incremental features across the five-week timeline.

6 Evaluation Criterion

... measurable and attributable

We define success using measurable metrics tied to the core workflow.

6.1 Primary evaluation metric

Time-to-verified-attendance (TTVA): time between photo upload and teacher-confirmed attendance record creation.

- Target: median TTVA ≤ 60 seconds per session during pilot testing.
- Attribution: measured from database timestamps (upload time vs. confirmation/save time).

6.2 Secondary evaluation metrics

- **Recognition accuracy:** percentage of students correctly identified without manual correction, compared against ground-truth roll-call.
- **Correction rate:** percentage of recognized entries manually corrected by the teacher, tracked to gauge model reliability over time.
- **Teacher-reported clarity:** simple 1–5 feedback question on ease of use of the verification interface.
- **Fallback resolution time:** time taken to resolve all unmatched students after face recognition, via NFC tap or manual add, measured as part of the overall verification step.
- **Reliability:** system availability for login, upload, and verification during class hours (target $\geq 99\%$ uptime in pilot).

6.3 Pilot validation plan

... *high-level*

- Pilot with 2–3 classrooms and their respective teachers over multiple sessions.
- Compare time spent and accuracy against manual roll-call, using teacher-reported baseline estimates for comparison.

7 Scalability

... *with foresight*

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in number of classrooms, students, and concurrent teacher usage by:
 - decoupling components (frontend/backend/recognition module),
 - enabling indexed queries for attendance history and analytics,
 - using stateless, token-authenticated API design.
2. **Operationally deployable (practically):** The system should run on common, low-cost cloud infrastructure:
 - a single managed compute instance for backend and database,
 - object storage for photographs,
 - CI/CD pipeline for staging/production deployment.

8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project:

- Web framework (FastAPI) for REST APIs, and React for the frontend.
- Managed relational database (PostgreSQL).
- Token-based authentication (JWT) with bcrypt password hashing.
- Object storage (Amazon S3) for classroom and registration photographs.
- An established open-source face-recognition/embedding library.
- Browser-native **Web NFC API** (NDEFReader, Chrome for Android) for the library-card fallback tap during verification, and the browser **Notification API** for confirming a successful tap – no dedicated NFC SDK or native app required.
- Test framework for backend unit/integration tests, and a CI runner for staging deployment.

We will use these mature components to focus on workflow design, correctness, and measurement rather than inventing new infrastructure, recognition algorithms, or NFC protocols.

9 Project scope and deliverables

... *iteration-friendly*

9.1 Initial deliverable

... *first iteration*

- Authentication and role-based dashboards (teacher, student, admin)
- Classroom listing and attendance session creation
- Photo upload and face recognition integration
- Manual verification interface with confirm/correct/add-manually actions, including the NFC fallback tap for unmatched students
- One-time NFC enrollment flow (tap-to-capture UID → map to roll number)
- Basic logging and timestamps for TTVA and fallback-resolution-time measurement
- CI: build + backend unit tests + linting

9.2 Subsequent deliverables

- Attendance history and CSV export for teachers/admins
- Analytics dashboard (resolution/backlog-style summaries: attendance trends, low-attendance alerts)
- AWS deployment (Vercel hosting for frontend, EC2 for backend and database)
- Improved indexing/query performance for scale readiness

10 Risks and mitigations

- **Recognition accuracy in poor lighting/crowded photos:** mandatory manual verification step ensures no attendance record is finalized without teacher confirmation.
- **Limited project timeline (4–5 weeks):** Agile iterative approach with weekly deliverables and early integration testing.
- **AWS free-tier resource limits:** single EC2 instance hosting both backend and database to minimize resource usage.
- **Student privacy concerns over face data:** store only embeddings (not raw comparison data) beyond registration photos, and restrict access via role-based authentication.
- **Web NFC browser/device constraint:** Web NFC is currently supported only on Chrome for Android; since the tap is only used briefly for the occasional unmatched student during verification (not a primary, high-volume flow), a single designated Android phone is sufficient.
- **Lost/unregistered/shared library cards:** taps with no matching UID are surfaced to the teacher as unmatched rather than silently dropped, and can be resolved with the existing manual-add option.

11 Summary

This project proposes a deployable web platform to reduce the time and error associated with manual classroom attendance, using photo-based face recognition as the primary attendance mechanism and an NFC library-card tap as a lightweight fallback for closing gaps left by unrecognized students during the same verification step. It meets the course criteria by emphasizing time-to-value through rapid iterations, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria (especially time-to-verified-attendance and fallback resolution time). It remains focused on engineering workflow, integration, and deployment readiness rather than research-driven novelty in face recognition.